CS4475 Fall 2026
<Your name>
<Your email address>

Lab Report
Submitted on <date>

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

The getprocs() scans the process table, skips UNUSED entries, copies process information into the user array, and returns the process count. Process locks protect each entry and wait_lock protects the parent information.

Files Changed: kernel/pstat.h, kernel/proc.h, kernel/proc.c, kernel/defs.h, kernel/sysproc.c, kernel/syscall.h, kernel/syscall.c, user/usys.pl, user/user.h, user/ps.c, and Makefile

```
$ ps
pid     state           size    age     priority        ppid    name
1       sleeping        12288           0               0       init
2       sleeping        16384           0               1       sh
14      sleeping        12288           49              2       pexec
15      sleeping        16384           49              14      sh
26      running         16384           49              15      ps
$
```

The parent IDs show the process relationships.

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

```
$ pexec 0 prioritytest
pid     state           size    age     priority        ppid    name
1       sleeping        12288           0               0       init
2       sleeping        16384           0               1       sh
14      sleeping        12288           49              2       pexec
15      sleeping        16384           49              14      sh
28      sleeping        12288           0               15      pexec
29      sleeping        16384           17              28      prioritytest
30      running         16384           49              29      ps
prioritytest: PASS range, inheritance, isolation, ps, bad pointer
$ pexec 10 ps
pid     state           size    age     priority        ppid    name
1       sleeping        12288           0               0       init
2       sleeping        16384           0               1       sh
14      sleeping        12288           49              2       pexec
15      sleeping        16384           49              14      sh
31      sleeping        12288           10              15      pexec
32      running         16384           10              31      ps
$
```

A priority field was added to struct proc and struct pstat. getprocs() copies this field into each returned entry, and user/ps.c prints it. The allocproc() initializes priority to 0, and fork() copies the parent's priority. getpriority() returns the caller's priority. setpriority() accepts 0–49 and

returns -1 for invalid values. The Task 1 output shows pexec, its child shell, and ps at priority 49, demonstrating inheritance.

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

readytime is initialized in allocproc() and updated to the current tick whenever a process becomes runnable in userinit(), fork(), yield(), wakeup() or kill().

The age calculation is: age = current ticks – readytime

```
ps$
pid     state           size    age     priority        ppid    name
1       sleeping        12288           0               0       init
2       sleeping        16384           0               1       sh
14      sleeping        12288           49              2       pexec
15      sleeping        16384           49              14      sh
34      sleeping        12288           49              15      pexec
35      sleeping        16384           49              34      sh
38      runnable        12288   4       0               37      schedtest
37      sleeping        12288           49              1       schedtest
39      runnable        12288   0       10              37      schedtest
40      zombie          12288           20              37      schedtest
42      running         16384           49              35      ps
```

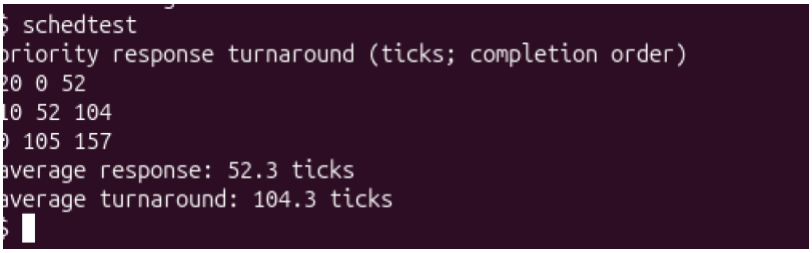
The ps output shows PID 38 as runnable with an age of 4 ticks and PID 39 as runnable with an age of 0 ticks. Sleeping, running, and zombie processes have blank age columns because they are not waiting in the ready queue.

Task 4. (15 pts) Implement a priority-based scheduler.

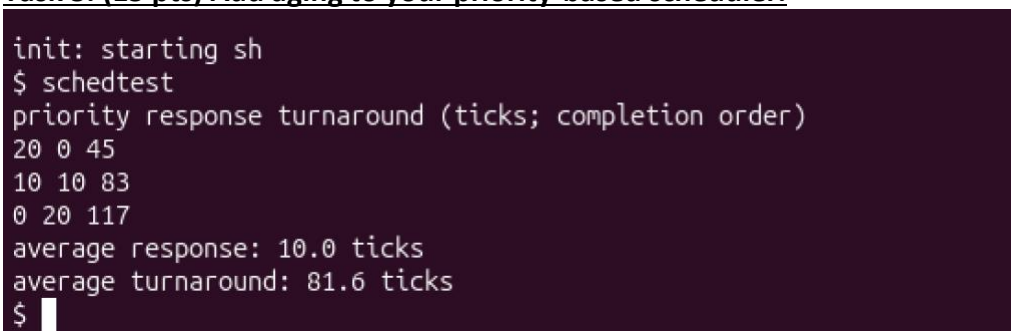
In kernel/param.h, I added constants to select the scheduler. In kernel/proc.c, I changed scheduler() to choose the runnable process with the highest priority. In Makefile, I selected one CPU and added the programs. user/schedtest.c tests scheduling, and user/pexec.c runs commands at a chosen priority.

The workers finished in priority order: 20, 10, then 0. Priority 20 started immediately, while priorities 10 and 0 waited 52 and 105 ticks. Average response time was 52.3 ticks, and average turnaround time was 104.3 ticks. This confirms that higher priority processes run first.

I learned that strict priority scheduling delays lower priority processes. Maintaining the correct locks was a difficulty, and the scheduler holds only the selected process's lock when switching.



Task 5. (15 pts) Add aging to your priority-based scheduler.



With aging, priority 10 started after 10 ticks and priority 0 after 20 ticks, compared with 52 and 105 ticks without aging. Average response decreased from 52.3 to 10.0 ticks, and average turnaround decreased from 104.3 to 81.6 ticks. Aging allowed lower priority workers to start before higher priority workers finished.

Task 6. (5 pts) Summary

I learned how system calls retrieve process information, how children inherit priorities, and how scheduling affects response and turnaround times. Aging helps lower priority processes get CPU time sooner. Difficulties included scheduler locking and compiler warnings. These were addressed through correct lock handling and compiler options. My priority test also needed to start at priority 0, which I handled with pexec 0 prioritytest.

OpenAI assisted with troubleshooting, and explanations. It also helped me use the UTM virtual machine, run the build and test commands, and understand the results.